

CO4-AT3

Name: N.R.Yasaswini

Register Number: 192425119

SET-2

1. Big Data Retail Analytics Engine

A global retail analytics engine must combine data generated from online purchases, physical stores, loyalty programs, social media, and inventory systems. A likely architecture uses an event-driven ingestion layer, a distributed data lake, scalable analytics engines, and separate serving databases.

Likely event ingestion pipeline:

- Point-of-sale systems, e-commerce applications, loyalty platforms, social media feeds, and inventory systems act as data sources.
- APIs, CDC (Change Data Capture), batch ETL, and message brokers such as Apache Kafka ingest events.
- Kafka topics can separate purchase, customer, inventory, and social-media events. Schema management ensures consistent event structures.
- Stream processors such as Apache Flink or Spark Structured Streaming clean, enrich, aggregate, and route events.

Distributed data lake:

- Raw data is stored in cloud object storage such as Amazon S3, Azure Data Lake Storage, or Google Cloud Storage.
- Data is organized into raw, cleaned, and curated zones.
- Parquet/ORC columnar formats and partitioning by date, region, product, or customer improve analytical performance.
- Spark or similar distributed engines process the data across many nodes.

Recommendation and segmentation:

Customer behavior is transformed into features such as purchase frequency, product affinity, recency, browsing activity, loyalty level, and response to promotions. Recommendation models may use collaborative filtering, content-based methods, or hybrid models. Customer segmentation can use clustering algorithms such as K-Means together with rule-based business segments. Distributed processing allows millions of customer-product interactions to be processed in parallel.

Likely databases:

- NoSQL databases such as Cassandra, DynamoDB, MongoDB, or HBase can provide low-latency access to customer profiles, sessions, recommendations, and operational data.
- Relational systems such as PostgreSQL, MySQL, or cloud data warehouses such as Snowflake, BigQuery, or Redshift can support structured reporting and historical analysis.

Caching and query optimization:

Frequently requested profiles, product information, recommendations, and dashboard results can be cached using Redis or Memcached. Query performance can be improved through indexes, partitioning, materialized views, columnar storage, query-result caching, compression, data clustering, and pre-aggregated tables.

Visualization and reporting:

BI tools such as Power BI, Tableau, or Looker can connect to curated analytical datasets. Executive dashboards may display sales, customer lifetime value, inventory turnover, campaign performance, regional trends, and forecasted demand. Managers can use these insights to optimize pricing, inventory, promotions, store operations, and customer engagement.

Overall, the likely design separates high-volume ingestion and distributed storage from low-latency serving systems while providing a governed analytical layer for strategic decision-making.

2. Global Logistics Company

A global logistics company needs to synchronize continuously changing shipment, warehouse, vehicle, and fuel information across regions. A likely solution uses cloud-native infrastructure, event streaming, distributed databases, and multi-region replication.

Likely cloud infrastructure:

- Applications and APIs run in containers or managed cloud services.
- Object storage forms the central data lake for historical shipment, inventory, fuel, and operational data.
- Kubernetes or managed compute services can scale processing workloads.
- Kafka or a cloud-managed equivalent provides global event ingestion.
- IAM, encryption, monitoring, and network controls protect the platform.

Distributed databases:

Operational shipment and inventory data may be stored in distributed NoSQL databases such as Cassandra, DynamoDB, or Cosmos DB for high availability and geographic distribution. Relational databases or cloud data warehouses can store structured financial, inventory, and historical reporting data.

GPS and inventory integration:

GPS devices continuously publish location events through a streaming layer. Stream-processing engines such as Flink or Spark Structured Streaming calculate current location, speed, estimated arrival time, and delay indicators. Inventory systems generate batch or CDC records. These records are cleaned and joined with streaming data using shipment IDs, vehicle IDs, warehouse IDs, and timestamps. The resulting operational view provides near-real-time visibility.

Analytics engines:

Spark, Flink, cloud-native SQL engines, and machine-learning platforms can support route optimization and demand forecasting. Historical delivery data can train models that estimate travel time, shipment demand, warehouse workload, and fuel consumption. Optimization algorithms then select routes based on distance, traffic, delivery windows, vehicle capacity, and fuel cost.

Event-driven architecture:

Events such as ShipmentCreated, PackageScanned, VehicleDeparted, WarehouseStockChanged, and DeliveryCompleted can trigger downstream services. For example, a warehouse scan can update shipment status, notify the customer system, and initiate transportation planning without tightly coupling all applications.

Redundancy and replication:

Critical services can be deployed across multiple availability zones and regions. Databases can use synchronous replication within a region and asynchronous cross-region replication. Object storage can use geo-redundancy and versioning. Load balancers, health checks, automated failover, backups, and disaster-recovery procedures reduce downtime.

Executive dashboards:

Operational pipelines feed curated KPI tables or semantic models. Dashboards can combine on-time delivery rate, shipment volume, warehouse utilization, fuel consumption, cost per shipment, inventory accuracy, and regional performance. Executives can therefore compare regions, identify bottlenecks, and make decisions about capacity, fleet allocation, warehouse operations, and customer service.

3. Big Data Cybersecurity Platform

A cybersecurity platform must analyze enormous volumes of network and endpoint events while detecting threats quickly. A probable architecture combines streaming ingestion, distributed stream processing, scalable log storage, machine learning, and automated incident response.

Streaming architecture:

Network devices, firewalls, authentication systems, servers, cloud services, EDR agents, and applications generate security events. Collectors forward these events to Kafka or a similar distributed message broker. Events can be separated into topics for login activity, firewall alerts, endpoint events, DNS traffic, and network flows.

Distributed processing:

Flink, Spark Structured Streaming, or a stream-processing/SIEM engine can normalize events, enrich them with threat intelligence, aggregate activity by user or IP address, and calculate real-time features. Rules can identify known attack patterns while machine-learning models detect unusual behavior.

Anomaly-detection training:

Historical security data is stored and labeled when confirmed incidents are available. Features may include login frequency, unusual locations, failed-login counts, access times, network destinations, data-transfer volumes, and endpoint behavior. Supervised models can learn from labeled attacks, while unsupervised methods such as clustering or isolation-based techniques can detect previously unseen anomalies. Real-time events are scored against these models, and confirmed incidents can be fed back for retraining.

Storage:

High-volume logs are typically stored in distributed object storage/data lakes using compressed formats for long-term retention. Hot security data can be maintained in Elasticsearch/OpenSearch, ClickHouse, or another fast search/analytics database. Time-based partitioning and lifecycle policies can move older data to cheaper storage.

Alerting and orchestration:

High-risk detections can create alerts in a SIEM or security operations platform. SOAR-style orchestration can automatically enrich an alert, check threat-intelligence sources, isolate an endpoint, disable a compromised account, or open an incident ticket. Human approval can be required for high-impact actions.

Security dashboards:

Dashboards aggregate authentication anomalies, malware detections, firewall activity, endpoint incidents, vulnerability indicators, and threat trends. Security teams can monitor incidents by severity, source, geography, affected assets, and time. This gives analysts a unified view even when underlying monitoring systems are distributed.

The architecture therefore combines fast streaming detection with large-scale historical analytics, allowing both immediate response and long-term threat hunting.

4. Online Education Platform

An online education platform generates structured student records as well as large multimedia and behavioral datasets. A likely architecture combines a data lake for raw and multimedia data with a data warehouse for curated educational analytics.

Data lake and warehouse:

Video events, quiz attempts, discussion posts, clickstream records, and application logs can be ingested into a cloud data lake. Video files are stored in scalable object storage, while metadata such as course ID, lesson ID, duration, language, and access permissions is stored separately. Structured student, course, enrollment, and assessment data can be stored in relational databases.

A warehouse or lakehouse contains cleaned and modeled datasets for reporting. ETL/ELT pipelines transform raw events into tables such as student activity, course progress, assessment performance, and attendance.

Recommendation models:

The platform can build learner profiles from course history, completed lessons, quiz performance, interests, search behavior, and content engagement. Collaborative filtering can recommend resources used successfully by similar learners, while content-based models can recommend materials matching a learner's interests and skill level. A hybrid approach can combine both. Distributed processing with Spark can calculate features and train models at large scale.

Live engagement and attendance:

Video plays, classroom joins, chat messages, quiz submissions, and interaction events can be sent through Kafka or another streaming platform. Flink or Spark Structured Streaming can calculate active learners, attendance duration, participation, and inactivity in near real time.

Multimedia and relational data:

Large video and audio files should be stored in object storage with CDN delivery and lifecycle policies. Relational databases store compact metadata and transactional student information. Links between the two are maintained using identifiers such as learner ID, course ID, and content ID rather than placing large media files inside relational tables.

Dropout prediction and performance monitoring:

Historical learner activity can be used to create features such as login frequency, missed assignments, declining engagement, quiz scores, and time since last activity. Machine-learning models can estimate dropout risk or identify learners who may need intervention. Dashboards can monitor model outputs and academic performance while respecting privacy and access controls.

Reporting and visualization:

Power BI, Tableau, Looker, or similar tools can provide dashboards for instructors and administrators. Instructors may view attendance, assessment performance, engagement, and at-risk learners. Administrators can analyze course completion, learner retention, content effectiveness, and institutional performance.

5. Big Data Transportation Platform

A real-time transportation platform combines GPS, ticketing, vehicle telemetry, and passenger-demand information. A suitable architecture requires high-throughput messaging, time-series storage, geospatial processing, cloud scalability, and real-time dashboards.

Messaging and ingestion:

Vehicles and mobile applications continuously generate GPS and telemetry events. Ticketing systems produce transactions, while passenger applications provide demand information. Kafka or cloud-managed streaming services can ingest these events into separate topics. Stream processors normalize timestamps, validate records, and enrich events with route, vehicle, and geographic information.

Distributed databases:

A distributed NoSQL database can store current vehicle states and operational records with low latency. Time-series databases are suitable for telemetry such as speed, engine status, fuel use, and temperature. Relational databases or cloud warehouses can maintain ticketing, financial, and historical reporting data. A data lake can retain raw events for long-term analysis.

Predictive analytics and optimization:

Historical GPS and ticketing data can train models to predict travel time, passenger demand, congestion, and delays. Forecasting models can estimate demand by route, time, and location. Optimization algorithms can use these predictions to adjust vehicle allocation, route selection, schedules, and fleet capacity. Real-time events can trigger dynamic rerouting when congestion or incidents occur.

Cloud infrastructure and replication:

The platform can run on a public cloud using managed compute, containers, object storage, databases, and streaming services. Services should be distributed across availability zones. Critical data can be replicated across regions, while backups and point-in-time recovery provide protection against data loss. Load balancing and automated failover keep services available during component failures.

Time-series and geospatial integration:

GPS coordinates are time-stamped, so they can be stored in time-series systems and queried over time windows. Geospatial databases or extensions can perform operations such as finding vehicles within an area, calculating distances, matching GPS points to routes, and identifying congestion zones. Combining geographic and temporal features enables accurate vehicle tracking and travel-time analysis.

Operational dashboards:

Transport authorities can use dashboards showing fleet locations, delays, passenger demand, route performance, ticket revenue, fuel consumption, and service reliability. Passenger-facing applications can expose estimated arrival times, vehicle locations, service alerts, and crowding information. These dashboards transform distributed real-time data into actionable operational information.

Overall, the architecture combines streaming and historical analytics so that transportation decisions can be made using both current conditions and long-term patterns.